ปฏิบัติการสัปดาห์ที่ 10: จัดทำเอกสาร API ด้วย OpenAPI/Swagger และ Postman/Newman

วัตถุประสงค์ของปฏิบัติการ

เมื่อทำปฏิบัติการนี้เสร็จสิ้น นิสิตจะสามารถ

1. เขียนไฟล์ OpenAPI Specification (`openapi.yaml`) บรรยาย endpoint จริงของโปรเจกต์ `student-api` ได้
2. ติดตั้ง Swagger UI ให้แสดงเอกสาร API ผ่านเบราว์เซอร์ และตรวจสอบว่าตรงกับพฤติกรรมจริงของโค้ด
3. สร้าง Postman Collection พร้อม Environment Variable สำหรับจัดการ token อัตโนมัติ
4. Export/Import Postman Collection และรันด้วย Newman CLI เพื่อทดสอบ API แบบอัตโนมัติ

ปฏิบัติการนี้ต่อยอดจากเนื้อหาบรรยายในเอกสาร `wk10.md` โดยขั้นตอนที่ 1-3 ฝึกเทคนิคทั้งหมดบนโปรเจกต์ตัวอย่าง `student-api` ที่สร้างต่อเนื่องมาจากปฏิบัติการสัปดาห์ที่ 1-9 เมื่อฝึกจนคล่องแล้ว นิสิตต้อง**ทำซ้ำขั้นตอนเดียวกันกับระบบโครงงานของตนเอง** (ระบบที่กำหนดไว้ในโครงงานระยะที่ 1 ตาม `wk05-project-phase1.md` เช่น ระบบหนังสือ ระบบงาน ระบบภาพยนตร์ ตามสูตร Pno ของแต่ละคน) เนื่องจากไฟล์ `openapi.yaml` และ `postman_collection.json` ที่ต้องส่งเป็นโครงงานระยะ 2 ส่วนที่ 1 (Docs/DX) ต้องบรรยาย endpoint ของ**ระบบโครงงานของตนเอง** ไม่ใช่ของ `student-api` ตามที่กำหนดไว้ใน `wk10-project-phase2.md` (วัดผล CLO6/PLO10 ตามที่กำหนดไว้ใน `course_plan.md`)

เวลาที่ใช้

ช่วงเวลา	กิจกรรม
0-15 นาที	ทบทวนเนื้อหาสัปดาห์ที่ 9 (Review W9 — Unit & Integration Testing)
15-90 นาที	เพิ่ม Swagger doc ตามโค้ดจริง
90-120 นาที	Export/Import Postman Collection และรัน Newman CLI

เครื่องมือที่ต้องใช้

1. โปรเจกต์ `student-api` จากปฏิบัติการสัปดาห์ที่ 1-9 (ต้องมี `app.js` / `server.js` แยกกันแล้วตามสัปดาห์ที่ 9 และมีชุดทดสอบ Jest/Supertest ทำงานได้)
2. Node.js และ npm
3. ไลบรารี `swagger-ui-express` , `js-yaml`
4. Postman (โปรแกรมหรือส่วนขยายเบราว์เซอร์)

5. Newman (`npm install -g newman`)

ส่วนที่ 1: ทบทวนเนื้อหาสัปดาห์ที่ 9 (0-15 นาที)

ขั้นตอนที่ 1.1 ตรวจสอบโปรเจกต์เดิม

เปิดโฟลเดอร์ `student-api` และรันชุดทดสอบเพื่อยืนยันว่าโค้ดจากสัปดาห์ที่ 1-9 ยังทำงานถูกต้องก่อนเริ่มเพิ่มเอกสาร

```
cd student-api
npm test
npm run dev
```

ขั้นตอนที่ 1.2 สำรวจ endpoint ทั้งหมดที่มีอยู่จริง

ก่อนเขียนเอกสาร ให้รวบรวมรายการ endpoint ทั้งหมดของโปรเจกต์จากไฟล์ route จริง (ไม่ใช่จากความจำ) เพื่อป้องกันเอกสารตกหล่นหรือไม่ตรงกับโค้ด บันทึกลงตารางชั่วคราว เช่น

Method	Path	ต้องยืนยันตัวตนหรือไม่
POST	`/api/v1/auth/register`	ไม่ต้อง
POST	`/api/v1/auth/login`	ไม่ต้อง
GET	`/api/v1/auth/me`	ต้อง
GET	`/api/v1/students`	ต้อง
POST	`/api/v1/students`	ต้อง (role: admin/teacher ตาม RBAC)
PUT	`/api/v1/students/:id`	ต้อง
DELETE	`/api/v1/students/:id`	ต้อง (role: admin)

จุดตรวจสอบที่ 1

- ชุดทดสอบ Jest/Supertest จากสัปดาห์ที่ 9 ยังรันผ่านทั้งหมด
- มีรายการ endpoint ครบถ้วนตรงกับโค้ดจริงในไฟล์ route (อย่างน้อย 6 endpoint ตามเกณฑ์ใน `wk10-project-phase2.md`)

ส่วนที่ 2: เพิ่ม Swagger Doc ตามโค้ดจริง (15-90 นาที)

ขั้นตอนที่ 2.1 ติดตั้งไลบรารี

```
npm install swagger-ui-express js-yaml
```

ขั้นตอนที่ 2.2 สร้างไฟล์ `openapi.yaml`

สร้างไฟล์ `openapi.yaml` ที่ระดับรากของโปรเจกต์ เริ่มจากโครงสร้างพื้นฐาน

```yaml
openapi: 3.0.3
info:
  title: Student API
  description: API สำหรับจัดการข้อมูลนิสิต พัฒนาในรายวิชา 88736065
  version: 1.0.0
servers:
  - url: http://localhost:3000/api/v1
    description: Development server
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    ErrorResponse:
      type: object
      properties:
        error:
          type: object
          properties:
            code:
              type: string
              example: "VALIDATION_ERROR"
            message:
              type: string
paths: {}
```

ขั้นตอนที่ 2.3 เพิ่ม path สำหรับ Authentication

เพิ่มเนื้อหาใน `paths` สำหรับ endpoint `POST /auth/register`, `POST /auth/login` และ `GET /auth/me` ตามตัวอย่างโครงสร้างในหัวข้อ 2.2 ของ `wk10.md` โดยอ้างอิงพฤติกรรมจริงจากโค้ดสัปดาห์ที่ 6 เช่น status code ที่ route จริงคืนกลับ (`201`, `400`, `401`, `409`)

ตรวจสอบให้ตรงกับโค้ดจริงโดยเปิดไฟล์ route ของ `auth` เทียบกับที่เขียนในเอกสาร เช่น หาก route คืน `409 DUPLICATE_EMAIL` เมื่อ email ซ้ำ (ตามที่ทดสอบไว้ใน `auth.test.js` ของสัปดาห์ที่ 9) เอกสารต้องระบุ response `409` นี้ไว้ด้วย ไม่ใช่ระบุเฉพาะ `201`

ขั้นตอนที่ 2.4 เพิ่ม path สำหรับ Students (CRUD)

เพิ่ม path สำหรับ `GET /students`, `POST /students`, `PUT /students/{id}` และ `DELETE /students/{id}` ตามตัวอย่างเต็มรูปแบบในหัวข้อ 2.2 ของ `wk10.md` โดยต้องระบุ

- `security: [{ bearerAuth: [] }]` สำหรับทุก endpoint ที่ต้องยืนยันตัวตน
- Query parameter `page` และ `limit` สำหรับ `GET /students` ตาม Pagination ที่ทำไว้ในสัปดาห์ที่ 7
- Response `403` สำหรับ endpoint ที่จำกัดสิทธิ์ตาม role (RBAC จากสัปดาห์ที่ 6) เช่น `DELETE /students/{id}`

ขั้นตอนที่ 2.5 เชื่อม Swagger UI เข้ากับแอปพลิเคชัน

แก้ไข `app.js`

```javascript
const swaggerUi = require("swagger-ui-express");
const YAML = require("js-yaml");
const fs = require("fs");
const path = require("path");

const openapiDocument = YAML.load(
  fs.readFileSync(path.join(__dirname, "openapi.yaml"), "utf8"),
);

app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(openapiDocument));
```

รันเซิร์ฟเวอร์แล้วเปิด `http://localhost:3000/api-docs`

```
npm run dev
```

ตรวจสอบว่า

1. หน้า Swagger UI แสดง endpoint ทั้งหมดที่เขียนไว้โดยไม่มี error ในการโหลดไฟล์ YAML
2. ทดลองกดปุ่ม **Try it out** ในหน้า Swagger UI เรียก `GET /students` และยืนยันว่าได้ response ตรงกับ schema ที่ประกาศไว้

จุดตรวจสอบที่ 2

- ไฟล์ `openapi.yaml` ครอบคลุม endpoint อย่างน้อย 6 endpoint ทุก resource หลักของโปรเจกต์
- แต่ละ endpoint ระบุ status code ที่เป็นไปได้ครบทุกกรณี ไม่ใช่เฉพาะกรณีสำเร็จ
- เปิด `/api-docs` แล้วแสดงผลถูกต้อง ไม่มี error จากการโหลด YAML
- ทดลองเรียก endpoint จริงผ่านปุ่ม Try it out ได้ผลลัพธ์ตรงกับที่เอกสารระบุ

ส่วนที่ 3: Export/Import Postman Collection และรัน Newman CLI (90-120 นาที)

ขั้นตอนที่ 3.1 จัดกลุ่ม Request ใน Postman เป็น Collection

เปิด Postman แล้วสร้าง Collection ชื่อ `Student API` จัดกลุ่มเป็นโฟลเดอร์ `Auth`, `Students` ตามโครงสร้างในหัวข้อ 4.2 ของ `wk10.md` สร้าง request ใหม่ในแต่ละโฟลเดอร์ให้ครบตาม endpoint ที่สำรวจไว้ในตารางของขั้นตอนที่ 1.2 (นิสิตไม่จำเป็นต้องมี request เก่าที่บันทึกไว้ตั้งแต่ก่อนหน้านี้ เพราะปฏิบัติการสัปดาห์ก่อน ๆ ทดสอบผ่าน Postman แบบเรียกทีละครั้งโดยไม่ได้บันทึกเก็บไว้)

ขั้นตอนที่ 3.2 สร้าง Environment และตัวแปร

สร้าง Environment ชื่อ `student-api-dev` พร้อมตัวแปร

ตัวแปร	ค่าเริ่มต้น
`baseUrl`	`http://localhost:3000/api/v1`
`authToken`	(เว้นว่างไว้ จะถูกตั้งค่าอัตโนมัติ)

แก้ไข URL ของทุก request ใน Collection ให้ใช้ `{{baseUrl}}` แทนการพิมพ์ URL โดยตรง

ขั้นตอนที่ 3.3 เพิ่มสคริปต์บันทึก Token อัตโนมัติ

เปิด request `POST Login` ไปที่แท็บ **Scripts > Post-response** (Postman เวอร์ชันใหม่เปลี่ยนชื่อจาก **Tests**) แล้วเพิ่ม

```javascript
const response = pm.response.json();
pm.environment.set("authToken", response.token);

pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
```

```
});

pm.test("Response has token", function () {
  pm.expect(response).to.have.property("token");
});
```

จากนั้นตั้งค่า Header ของ request ที่ต้องยืนยันตัวตน (เช่น `GET Me`, `POST Create Student`) เป็น `Authorization: Bearer {{authToken}}`

ขั้นตอนที่ 3.4 เพิ่ม Test Script ให้ทุก Request

เพิ่มการทดสอบพื้นฐานในแท็บ **Scripts > Post-response** ของแต่ละ request อย่างน้อยตรวจสอบ status code ที่ถูกต้อง เช่น

```javascript
// GET Students - Scripts > Post-response
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response has pagination info", function () {
  const body = pm.response.json();
  pm.expect(body).to.have.property("pagination");
});
```

ขั้นตอนที่ 3.5 Export Collection และ Environment

```
Collection ... (เมนู) > Export > Collection v2.1 (recommended) → บันทึกเป็น
postman_collection.json
Environment > ... (เมนู) > Export → บันทึกเป็น environment.json
```

ทดสอบ Import กลับเข้า Postman อีกครั้งในหน้าต่างใหม่ (หรือให้เพื่อนร่วมกลุ่มลอง import) เพื่อยืนยันว่าไฟล์ที่ export ใช้งานได้จริงโดยไม่ต้องตั้งค่าเพิ่มเติม

ขั้นตอนที่ 3.6 รัน Newman CLI

```
npm install -g newman
newman run postman_collection.json -e environment.json
```

ตรวจสอบผลลัพธ์ในตารางสรุปท้ายรายงาน คอลัมน์ `failed` ของทุกแถวต้องเป็น `0`

```
┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │           executed │             failed │
```

```
├──────────────────────────┼──────────────────┼──────────────────┤
│              iterations  │                1 │                0 │
│                requests  │                7 │                0 │
│            test-scripts  │                7 │                0 │
│              assertions  │               14 │                0 │
└──────────────────────────┴──────────────────┴──────────────────┘
```

หากมี request ใดล้มเหลว ให้ตรวจสอบลำดับการรัน (Newman รันตามลำดับที่ปรากฏใน Collection) โดยเฉพาะว่า `POST Login` ถูกรันก่อน request ที่ต้องใช้ `{{authToken}}` หรือไม่

จุดตรวจสอบที่ 3

- Export ได้ไฟล์ `postman_collection.json` และ `environment.json` และ import กลับมาใช้งานได้จริง
- ทุก request ที่ต้องยืนยันตัวตนใช้ `{{authToken}}` ที่ตั้งค่าอัตโนมัติจากสคริปต์ของ `POST Login` โดยไม่ต้องคัดลอก token ด้วยตนเอง
- รัน `newman run` แล้วทุก assertion ผ่านโดยไม่มีค่า `failed` มากกว่า 0

ส่วนต่อยอด: แบบฝึกหัดเพิ่มเติมด้วยตนเอง

แบบฝึกหัดที่ 1: เพิ่ม Path Parameter Description ให้ครบทุก Endpoint

ตรวจสอบไฟล์ `openapi.yaml` ว่าทุก path parameter (เช่น `{id}`) มีคำอธิบาย (`description`) และตัวอย่างค่า (`example`) ครบถ้วน ไม่ใช่ระบุเพียง `type` เท่านั้น

แบบฝึกหัดที่ 2: ระบุพฤติกรรม Caching ใน OpenAPI

`GET /students` ของโปรเจกต์มีกลไก Caching อยู่เบื้องหลังตามที่ทำไว้ในสัปดาห์ที่ 7 (ไม่ใช่ endpoint แยกต่างหาก) ให้เพิ่มคำอธิบายพฤติกรรมนี้ไว้ใน `description` ของ path `GET /students` ในไฟล์ `openapi.yaml` พร้อมระบุ response header ที่ระบบใช้แจ้งสถานะ cache hit/miss (ถ้ามี) ไว้ในเอกสาร response ของ endpoint นั้น

แบบฝึกหัดที่ 3: เพิ่ม Pre-request Script ตรวจสอบ Environment

เพิ่ม Pre-request Script ระดับ Collection ใน Postman เพื่อตรวจสอบว่าตัวแปร `baseUrl` ถูกตั้งค่าไว้แล้วก่อนเรียก request ใด ๆ หากไม่ได้ตั้งค่าให้แสดง error ที่ชัดเจนใน console ของ Postman

แบบฝึกหัดที่ 4: ทดลอง Generate Client Code จาก OpenAPI

ใช้เครื่องมือ Swagger Editor (https://editor.swagger.io) วางเนื้อหาไฟล์ `openapi.yaml` แล้วทดลองใช้ฟีเจอร์ Generate Client เพื่อสร้างโค้ดตัวอย่างสำหรับเรียก API ด้วยภาษาใดภาษาหนึ่ง บันทึกผลลัพธ์และอธิบายว่าโค้ดที่ได้ตรงกับ endpoint จริงหรือไม่ อย่างไร

แบบฝึกหัดที่ 5: รัน Newman พร้อมสร้างรายงาน HTML

ติดตั้งส่วนขยายของ Newman เพื่อสร้างรายงานในรูปแบบ HTML แทนการแสดงผลเฉพาะใน terminal

```
npm install -g newman-reporter-htmlextra
newman run postman_collection.json -e environment.json -r htmlextra
```

เปิดไฟล์รายงานที่ได้และอธิบายว่าข้อมูลใดในรายงานมีประโยชน์มากกว่าการอ่านผลลัพธ์จาก terminal

เกณฑ์การประเมินแบบฝึกหัดต่อยอด (สำหรับนิสิตใช้ตรวจสอบตนเอง)

รายการ	ผ่านเกณฑ์เมื่อ
แบบฝึกหัดที่ 1	ทุก path parameter มี `description` และ `example` ครบถ้วนใน `openapi.yaml`
แบบฝึกหัดที่ 2	`description` ของ path `GET /students` ใน OpenAPI ระบุพฤติกรรม caching พร้อมรายละเอียด header ที่เกี่ยวข้อง
แบบฝึกหัดที่ 3	Pre-request Script ตรวจสอบและแจ้งเตือนเมื่อ `baseUrl` ไม่ถูกตั้งค่า ทดสอบแล้วทำงานจริง
แบบฝึกหัดที่ 4	มีบันทึกผลลัพธ์จาก Swagger Editor พร้อมคำอธิบายเปรียบเทียบกับ endpoint จริง
แบบฝึกหัดที่ 5	รันได้รายงาน HTML สำเร็จ พร้อมคำอธิบายประโยชน์เทียบกับรายงานแบบ terminal

หมายเหตุสำคัญ: ขั้นตอนที่ 1-3 ข้างต้นฝึกบน `student-api` เพื่อให้เห็นตัวอย่างที่ถูกต้องก่อน แต่ไฟล์ที่ต้อง**ส่งจริง**เป็นโครงงานระยะ 2 ส่วนที่ 1 (Docs/DX) คือ `openapi.yaml` และ `postman_collection.json` ของ**ระบบโครงงานของตนเอง** (ไม่ใช่ของ `student-api`) ตามรายละเอียดเกณฑ์การให้คะแนนใน `wk10-project-phase2.md` และจะถูกนำไปใช้ต่อในสัปดาห์ที่ 11 (External API Integration & Resilience) ซึ่งจะเพิ่มการเรียกใช้ API ภายนอกและกลไก Retry/Timeout/Fallback เข้าไปในระบบโครงงานเดียวกันนี้